

```
from tasks import email

@app.route('/order/add', method=['POST'])
def submit_order():
    # create entries in your database

    email.delay(g.user['email'], 'Order confirmation.',
               '<some message>')

    return render_template('submit_form.html')
```

In the above code, all we are doing is importing the task, which is essentially a method (*email* method) in the *tasks* module, and we queue it up using the *delay* method on this task, passing the required parameters.

```
celery -A tasks worker --loglevel=info
```

We use the *celery* command to start a worker by passing the worker argument and the name of the module that contains our tasks—in this case, *tasks*. You may also run the worker in the background, using a *celery* option. That's it! Celery is that simple to use! This also shows that if you already have isolated independent code in the form of methods, you can easily convert them into tasks using the *@task* decorator.

As I discussed earlier, Celery can also be used to schedule tasks and repeating jobs like *cron*. To do something like that, this is what you have got to do:

```
from celery.schedules import crontab

CELERYBEAT_SCHEDULE = {
    'email-every-ten-minutes': {
        'task': 'tasks.email',
        'schedule': timedelta(minutes=10),
        'args': ('admin@mysite.com', 'Site Health',
                 '<some awesome stats>'),
    },
}
```

In the above example, we are basically adding a configuration for Celery; the name of the configuration is *CELERYBEAT_SCHEDULE*. *Celery Beat* is the scheduler that comes with Celery. The *CELERYBEAT_SCHEDULE* configuration is just a Python dictionary, with the key as its name, and the value is another Python dictionary, with various fields for defining the scheduled task. The task field requires the name of the task to execute. In our case, it's *tasks.email* because the email method is in *tasks.py* (the *tasks* module). The *schedule* field requires a value defining the frequency of execution. Here, we have defined it to be 10 minutes (as a simple example), but this can be as complex as executing a task on every first month of

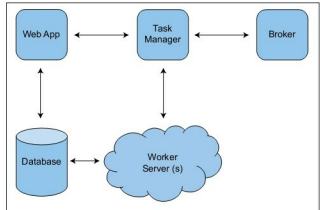


Figure 3: In production

every quarter of the year. To have your scheduled tasks running, just run your workers as we saw earlier, and then run the command *celery beat*. With this, you will have your repeated tasks running in no time.

Celery is amazingly useful. We saw some of its out-of-the-box features, but there is a lot more that it has to offer. Check out <http://celeryproject.org/>, as it has the most amazing documentation for Celery, and is the one-stop-shop for it.

MQ solutions and protocols

There are a lot of MQ solutions or MQ Brokers out there—RabbitMQ, Amazon Simple Queue, Apache MQ, Gearman, Starling, OpenAMQ, Sun Java Message Queue System... and this list is definitely not exhaustive. To my knowledge, RabbitMQ is a pretty famous solution, backed by VMWare, and is used a lot (at least in the Python community). You can definitely choose any one you want. There are also some competing protocols for message queuing:

- Asynchronous Message Queue Protocol (AMQP): RabbitMQ implements this protocol.
- Java Messaging Service (JMS)
- Streaming Text Oriented Messaging Protocol (STOMP)

We are not going to discuss protocols any further, because that requires going very deep into the topic. But you normally won't need to go too much into the subject—and when you need to, you can do your research.

Criteria for broker selection

Since there is a wide variety of brokers available, it might get difficult for you to zero in on one broker to include in your stack. Also, your selection process is vital, as you are adding a new system in your stack, which means that it will require some amount of work to get it in. Therefore, here are a few parameters that you ought to consider while selecting a broker:

- Difficulty in recovery: How difficult is it to recover your MQ system if something breaks or crashes?

Continued on Page 75...